PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DBMS PROJECT REPORT

SMART RESEARCH & PEER REVIEW PORTAL

Submitted by

PRASHANT DEVENDRA NAIK
SRN: PES1UG24AM199

PRADYUN P URS
SRN: PES1UG24AM195

Academic Year 2026–2027

TABLE OF CONTENTS

- 1. Project Title and Details
- 2. Abstract
- 3. Introduction
- 4. Problem Statement
- 5. Objectives and Scope
- 6. Proposed System and Workflow
- 7. Technology Stack
- 8. System Architecture
- 9. Database Design
- 10. Entity and Relationship Design
- 11. PostgreSQL – SQL Database Commands
- 12. SQL Concepts Used
- 13. Functions, Triggers, Transactions and Joins
- 14. NoSQL and Vector Database
- 15. Next.js Project Tree Structure
- 16. Application Modules and API Description
- 17. Student Review Dashboard
- 18. Testing and Validation
- 19. Screenshots and Evidence
- 20. Evaluation Rubrics – Completion Status
- 21. Limitations and Future Enhancements
- 22. Conclusion
- 23. References

Note: Update page numbers after inserting screenshots and generating the final table of contents.

1. PROJECT TITLE AND DETAILS

Field	Details
Project Title	Smart Research & Peer Review Portal
Subtitle	A Hybrid SQL and NoSQL-Based Academic Research Management System
Student 1	PRASHANT DEVENDRA NAIK – PES1UG24AM199
Student 2	PRADYUN P URS – PES1UG24AM195
Institution	PES University
Department	Computer Science and Engineering
Academic Year	2026–2027
Primary Database	PostgreSQL 18
Application Framework	Next.js App Router with TypeScript

2. ABSTRACT

The Smart Research & Peer Review Portal is an academic research management application intended to organize the process of research manuscript submission, reviewer allocation, manuscript evaluation, and feedback delivery. The system provides separate application capabilities for students, reviewers, and administrative users.

The relational database is implemented using PostgreSQL. It stores structured information about departments, users, research topics, manuscripts, reviewer profiles, assignments, reviews, and audit logs. The Next.js application communicates with PostgreSQL through the Node.js pg connection pool. SQL queries are used to retrieve user information, connect related records, and display review results.

The review workflow connects a student's manuscript to reviewer assignments and submitted reviews. Once a reviewer submits an evaluation, the student review page can display technical score, originality score, average score, reviewer comments, recommendation, and review date. Access to the student's reviews is filtered using the authenticated session.

The project demonstrates relational database concepts such as primary keys, foreign keys, constraints, joins, aggregate calculations, views, stored functions, triggers, and audit logging. MongoDB and pgvector were considered as parts of a hybrid data approach; their operational status must be demonstrated separately and is not claimed as complete in this report.

3. INTRODUCTION

Research paper evaluation involves multiple participants and several related activities. A student prepares and submits a manuscript, an administrator allocates the manuscript to a suitable reviewer, and the reviewer evaluates the work and submits feedback. If these activities are managed separately, it can become difficult to track manuscript status, assignments, review history, and feedback.

The Smart Research & Peer Review Portal is designed to bring these activities into one web application. The database maintains relationships among users, manuscripts, reviewers, assignments, and reviews. The application uses a browser-based interface and backend API routes to perform database operations.

4. PROBLEM STATEMENT

The project addresses the need for a centralized system to manage academic research submissions and peer-review activities. The system must maintain manuscript information, identify the student who submitted each manuscript, record reviewer assignments, store evaluation results, and allow the correct student to view feedback. It must also preserve data integrity through database keys and constraints and maintain an audit record of manuscript status changes.

5. OBJECTIVES AND SCOPE

5.1 Objectives

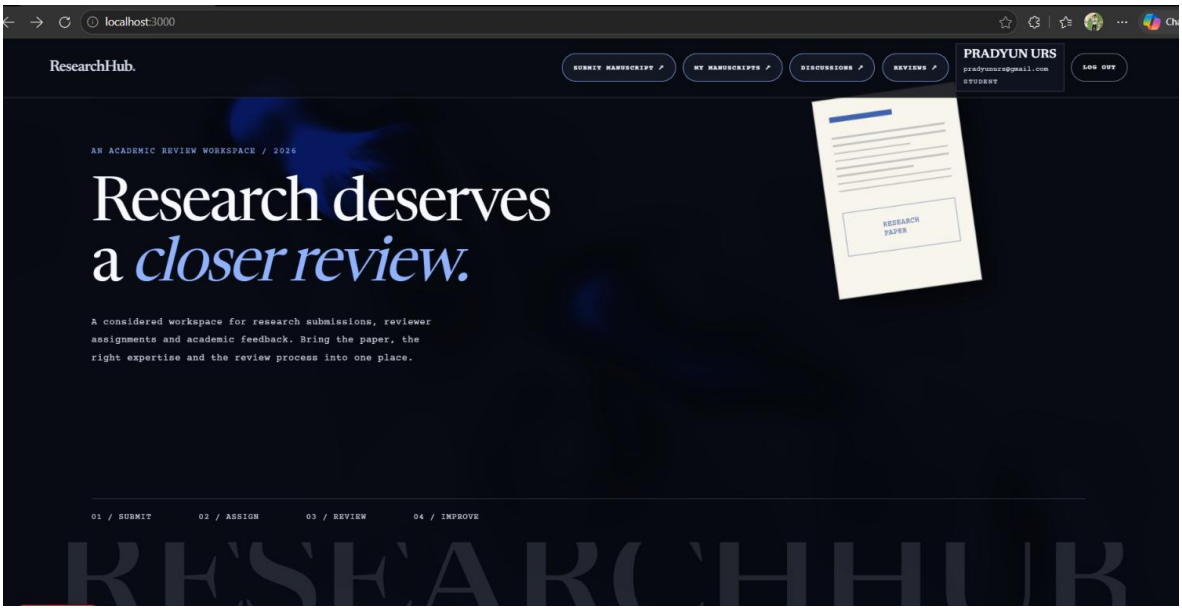
- Provide a central portal for academic manuscript submission and review.
- Maintain student, reviewer, administrator, department, and research-topic information.
- Connect manuscripts with assigned reviewers.
- Store technical scores, originality scores, comments, and recommendations.
- Allow students to view evaluations associated with their own manuscripts.
- Use relational database constraints to maintain data integrity.
- Demonstrate SQL joins, functions, triggers, views, and audit logging.

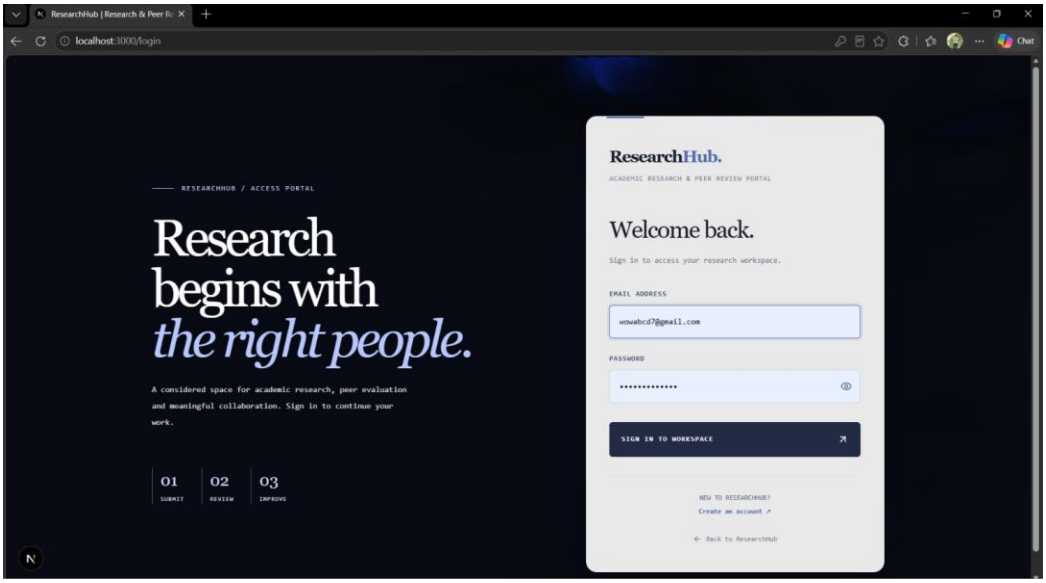
5.2 Scope

The confirmed relational scope includes the PostgreSQL schema and its nine physical tables. The Next.js application includes authentication-related code and a student review page/API route. Features such as MongoDB-backed discussions and pgvector semantic search should be included in the completed scope only after they are verified in the running application.

6. PROPOSED SYSTEM AND WORKFLOW

1. Student signs in and submits a manuscript.
2. Manuscript details are stored in the manuscripts table and linked to the student through student_id.
3. An administrator or coordinator creates an assignment connecting a manuscript and reviewer.
4. Reviewer reads the manuscript and evaluates it.
5. Reviewer submits scores, comments, and a recommendation.
6. Review details are stored in the reviews table and linked to the assignment.
7. Student dashboard retrieves reviews by following the relationships and filtering by the authenticated student's user ID.





7. TECHNOLOGY STACK

Technology	Role in the Project
Next.js App Router	Frontend pages and backend API route handling
React	Interactive user interface
TypeScript	Typed application logic
Tailwind CSS	Styling and responsive layout
Node.js	Server-side JavaScript runtime
PostgreSQL 18	Relational database
pg	PostgreSQL connection pooling and SQL execution
jose	JWT session creation and verification in auth.ts
bcrypt/bcryptjs	Password verification if used by the current login

	implementation
MongoDB	Planned/additional document-oriented database; verify integration
pgvector	Planned vector similarity extension; installation was previously blocked

8. SYSTEM ARCHITECTURE

The application follows a web application architecture. The browser displays Next.js pages. Backend route handlers process requests, verify authentication and roles, and execute parameterized SQL through the PostgreSQL connection pool. PostgreSQL stores the structured application records.

```
Browser / Student or Reviewer
          |
          v
Next.js pages (React + TypeScript)
          |
          v
Next.js API route handlers
          |
          +---- Authentication/session verification (lib/auth.ts)
          |
          v
PostgreSQL connection pool (lib/db.ts)
          |
          v
PostgreSQL database (smartportal_db)
```

The database connection file exports a Pool instance from the pg package. The authentication file uses a signed session token stored in an HTTP-only cookie. The API should verify the session and role before returning protected data.

9. DATABASE DESIGN

9.1 Database Details

Property	Value
DBMS	PostgreSQL
Database name	smartportal_db
Known PostgreSQL version	18.6
Physical tables	9
Known view	manuscript_review_summary

9.2 Table Descriptions

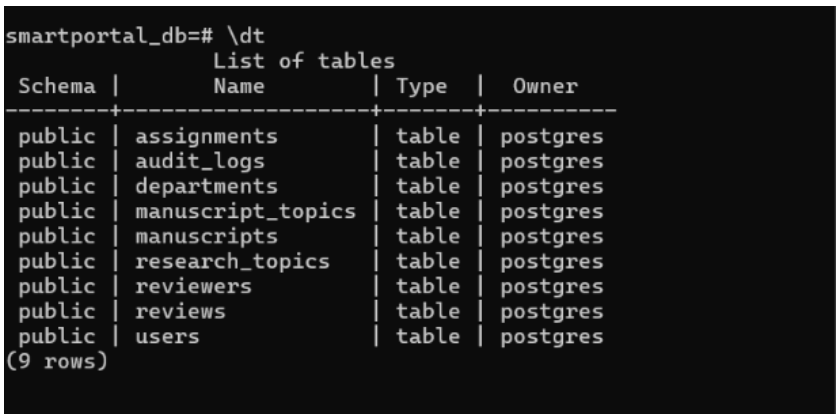
Table	Main purpose
departments	Stores department names and codes.

users	Stores account details, role, email, password hash, and optional department.
research_topics	Stores research topic names and descriptions.
manuscripts	Stores title, abstract, keywords, student owner, status, and timestamps.
reviewers	Links reviewer profiles to user accounts and stores expertise.
manuscript_topics	Associates manuscripts with research topics using a composite key.
assignments	Links a manuscript with an assigned reviewer and due date/status.
reviews	Stores review comments, technical score, originality score, recommendation, and review time.
audit_logs	Stores recorded actions and manuscript status-change history.

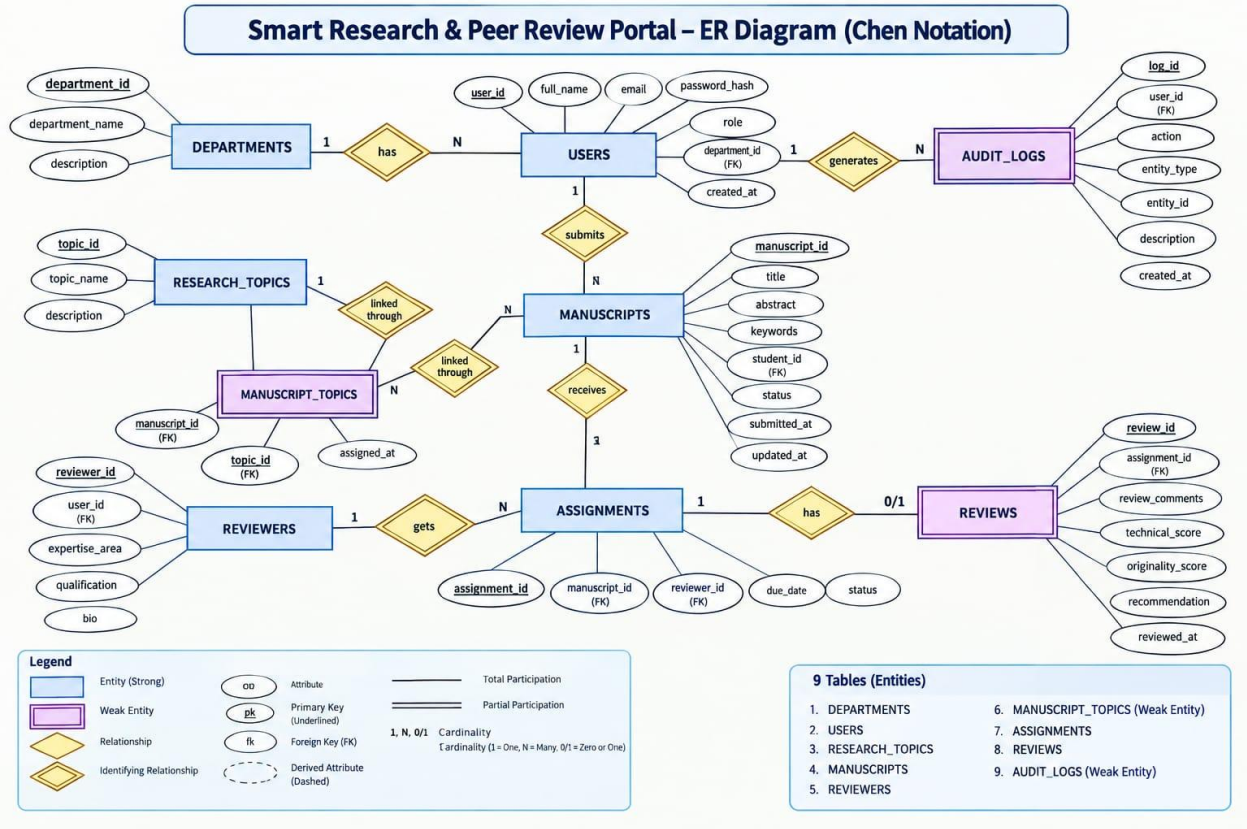
9.3 Relationships and Foreign Keys

Child table and column	Referenced table and column
users.department_id	departments.department_id
manuscripts.student_id	users.user_id
reviewers.user_id	users.user_id
assignments.manuscript_id	manuscripts.manuscript_id
assignments.reviewer_id	reviewers.reviewer_id
reviews.assignment_id	assignments.assignment_id
manuscript_topics.manuscript_id	manuscripts.manuscript_id
manuscript_topics.topic_id	research_topics.topic_id
audit_logs.user_id	users.user_id

The manuscript_topics table uses a composite primary key consisting of manuscript_id and topic_id. Known unique constraints include users.email, departments.department_name, departments.department_code, research_topics.topic_name, reviewers.user_id, assignments (manuscript_id, reviewer_id), and reviews.assignment_id.



```
smartportal_db=# \dt
                List of tables
 Schema |       Name        | Type  |  Owner
--------+-------------------+-------+----------
 public | assignments       | table | postgres
 public | audit_logs        | table | postgres
 public | departments       | table | postgres
 public | manuscript_topics | table | postgres
 public | manuscripts       | table | postgres
 public | research_topics   | table | postgres
 public | reviewers         | table | postgres
 public | reviews           | table | postgres
 public | users             | table | postgres
(9 rows)
```



10. ENTITY AND RELATIONSHIP DESIGN

The database is organized around users, manuscripts, reviewers, assignments, and reviews. A user can submit multiple manuscripts. Each manuscript belongs to one student account through manuscripts.student_id. A manuscript can have reviewer assignments. Each assignment connects one manuscript to one reviewer. A submitted review is connected to its assignment through reviews.assignment_id.

The manuscript_topics table resolves the many-to-many relationship between manuscripts and research_topics. The composite primary key prevents the same topic association from being entered more than once for the same manuscript.

The audit_logs table records system actions. The known manuscript status trigger inserts a log entry when a manuscript status is updated.

11. POSTGRESQL – SQL DATABASE COMMANDS

11.1 Connect to the database

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -d smartportal_db
```

11.2 List tables and inspect schema

```
\dt
\d users
\d manuscripts
```

```
\d assignments
\d reviews
```

11.3 SELECT examples

```
SELECT * FROM users;
SELECT * FROM manuscripts;
SELECT * FROM assignments;
SELECT * FROM reviews;
SELECT * FROM audit_logs;
```

11.4 INSERT example

```
INSERT INTO research_topics (topic_name, description)
VALUES ('Artificial Intelligence', 'Research related to AI');
```

Use the actual column definitions and existing constraints when inserting records. Identity columns generated ALWAYS should normally be omitted from INSERT statements.

11.5 UPDATE example

```
UPDATE manuscripts
SET status = 'UNDER_REVIEW'
WHERE manuscript_id = 1;
```

The manuscript status trigger is expected to record this status change in audit_logs.

11.6 JOIN example

```
SELECT m.title, u.full_name, m.status
FROM manuscripts AS m
INNER JOIN users AS u
    ON m.student_id = u.user_id;
```

11.7 GROUP BY and COUNT example

```
SELECT status, COUNT(*) AS total_manuscripts
FROM manuscripts
GROUP BY status;
```

11.8 Average review score example

```
SELECT AVG((technical_score + originality_score) / 2.0)
       AS average_review_score
FROM reviews
WHERE technical_score IS NOT NULL
  AND originality_score IS NOT NULL;
```

11.9 View and function inspection

```
\dv
\df
SELECT * FROM manuscript_review_summary;
```

12. SQL CONCEPTS USED

Concept	Explanation and project relevance
DDL – Data Definition Language	Defines database structures. Examples include CREATE TABLE and ALTER TABLE. Used when designing the

	PostgreSQL schema.
DML – Data Manipulation Language	Changes stored records using INSERT, UPDATE, and DELETE.
DQL – Data Query Language	Retrieves data using SELECT.
Primary key	Uniquely identifies each row, such as users.user_id.
Foreign key	Maintains relationships between tables, such as manuscripts.student_id referencing users.user_id.
UNIQUE constraint	Prevents duplicate values, such as duplicate user email addresses.
NOT NULL constraint	Requires important values to be present.
DEFAULT constraint	Provides a default value, such as CURRENT_TIMESTAMP.
JOIN	Combines related information from multiple tables. Used in the student review query.
Aggregate functions	COUNT and AVG calculate summaries such as manuscript counts and average scores.
GROUP BY	Groups records, for example by manuscript status.
View	A stored query exposed as a virtual table. The database contains manuscript_review_summary.
Stored function	Reusable PostgreSQL logic. The project has log_manuscript_status_change().
Trigger	Automatically invokes logic after a manuscript status update.
Transaction	Groups statements using BEGIN, COMMIT, and ROLLBACK. Explain application use only if verified.
Composite primary key	Uses manuscript_id and topic_id together in manuscript_topics.
Parameterized query	Uses placeholders such as $1 with pool.query to separate SQL from supplied values.
Audit logging	Records status-change actions in audit_logs.

13. FUNCTIONS, TRIGGERS, TRANSACTIONS AND JOINS

13.1 JOIN

JOIN retrieves related information from more than one table. The student review query joins reviews to assignments, manuscripts, reviewers, and users. This is necessary because scores are stored in reviews, manuscript titles are stored in manuscripts, and reviewer names are stored in users.

13.2 Function

The PostgreSQL function log_manuscript_status_change() contains reusable logic for recording a manuscript status change. It inserts a row into audit_logs.

13.3 Trigger

The trigger manuscript_status_audit_trigger is associated with the manuscripts table and runs after an UPDATE. It calls log_manuscript_status_change(). The known function inserts the log with user_id set to NULL; it does not currently identify the authenticated actor.

13.4 Transaction

A transaction groups database operations into a logical unit. BEGIN starts a transaction, COMMIT saves its changes, and ROLLBACK cancels uncommitted changes. This report demonstrates the SQL transaction concept; do not claim a particular application workflow uses explicit transaction blocks unless that code is verified.

```
BEGIN;

UPDATE manuscripts
SET status = 'UNDER_REVIEW'
WHERE manuscript_id = 1;

COMMIT;
```

13.5 ACID properties

Property	Meaning	Relevant example
Atomicity	All operations in a transaction succeed or none are committed.	A group of related database updates.
Consistency	Constraints and database rules remain valid.	Foreign-key and NOT NULL constraints.
Isolation	Concurrent transactions are controlled.	Concurrent application users.
Durability	Committed changes persist.	Saved review records.

14. NOSQL AND VECTOR DATABASE COMMANDS

The project was described as a hybrid SQL and NoSQL design. PostgreSQL is the confirmed relational database. MongoDB was considered for flexible document-oriented information such as discussions or messages. However, the actual MongoDB database, collection names, and completed application CRUD integration must be verified before this section is presented as completed implementation.

14.1 MongoDB verification commands

```
show dbs
use your_database_name
show collections
db.your_collection_name.find()
db.your_collection_name.findOne()
```

Replace the example database and collection names with the actual names shown in MongoDB Compass or mongosh. Include screenshots only after confirming that the application uses the collection.

14.2 pgvector status

The pgvector extension was previously blocked on the Windows environment by an application-control policy when PostgreSQL attempted to load vector.dll. Therefore, semantic vector search is not reported as

operational in this document. It can be marked complete only after CREATE EXTENSION vector succeeds and a vector search query is tested.

```
CREATE EXTENSION vector;
```

15. NEXT.JS PROJECT TREE STRUCTURE

```
dbmsminiproject/
├── app/
│   ├── api/
│   │   ├── auth/
│   │   │   └── login/
│   │   │       └── route.ts
│   │   └── student/
│   │       └── dashboard/
│   │           └── route.ts
│   └── student/
│       └── dashboard/
│           └── page.tsx
├── components/
├── lib/
│   ├── auth.ts
│   └── db.ts
├── public/
├── .env.local
├── middleware.ts
├── next.config.ts
├── package.json
└── README.md
```

This is the relevant structure discussed during implementation. Before submission, compare it with the actual project tree and update it to include any additional routes or components that are present.

16. APPLICATION MODULES AND API DESCRIPTION

Module / file	Responsibility
lib/db.ts	Creates and exports the PostgreSQL Pool using environment variables.
lib/auth.ts	Creates, verifies, reads, and destroys the signed session cookie.
Login API	Looks up a user account and verifies the submitted password according to the implementation.
Student dashboard API	Checks session and STUDENT role, then retrieves reviews for the session user.
Student dashboard page	Displays review cards, scores, comments, recommendation, and review date.

The session type contains userId, fullName, email, and role. The student dashboard API should use session.userId as the filter value. It should not trust a student ID supplied by the browser.

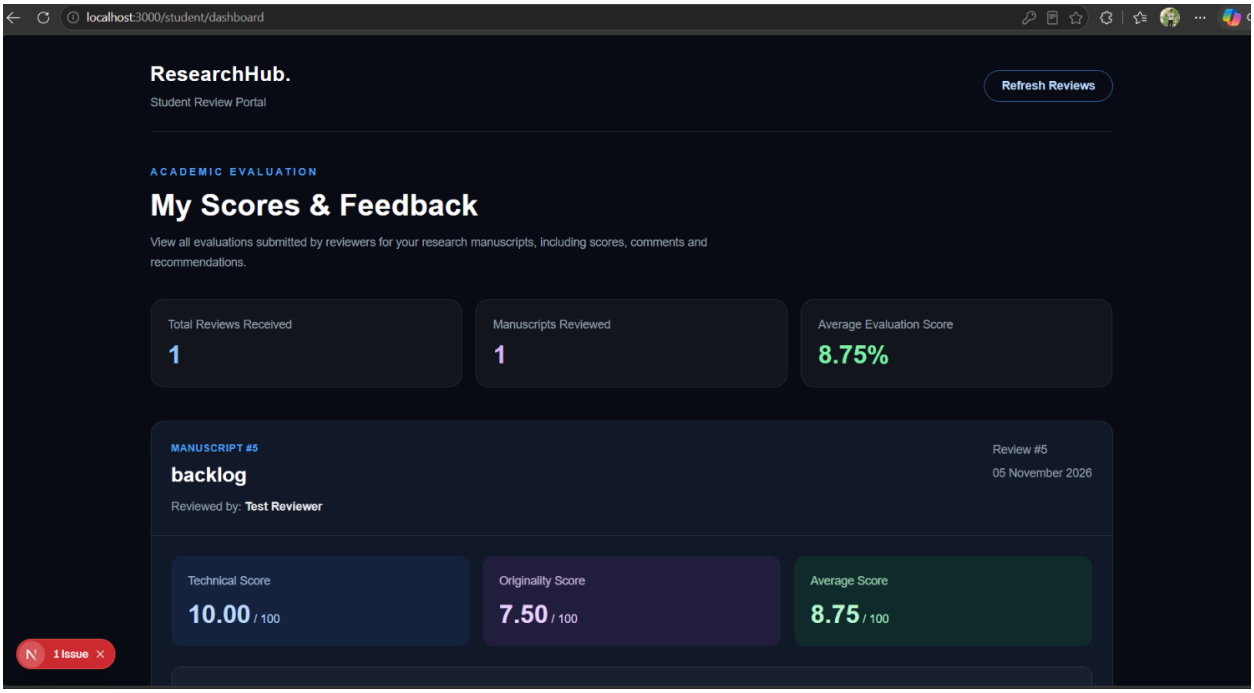
17. STUDENT REVIEW DASHBOARD

The student review page is a single-page view of submitted evaluations. It displays a summary of review count, number of manuscripts reviewed, and average evaluation score, followed by a review card for each submitted review.

Each review card can show manuscript title, reviewer name, technical score, originality score, average score, reviewer comments, recommendation, review date, and assignment identifier.

The page fetches data from /api/student/dashboard. If no review has been submitted, the page displays an empty-state message. Pending assignments are not shown as completed reviews.

The average displayed in the current implementation is calculated from the technical and originality scores. It should not be described as an official final grade unless the project defines that grading policy.



18. TESTING AND VALIDATION

Test	Expected result / evidence
PostgreSQL connection	The application can connect to smartportal_db.
Table verification	The nine expected physical tables appear in \dt.
Student login	A valid student can sign in.
Protected API	Unauthenticated request is rejected.
Role authorization	A non-student role cannot access the student review API.
Student-specific filtering	Only reviews belonging to the session student's manuscripts are returned.
Review retrieval	Submitted review records appear in the API response.
Dashboard display	Scores, comments, recommendation, and date display

	correctly.
Audit trigger	Updating manuscript status creates an audit_logs record.

Record the actual outcome of each test before submission. Do not mark a test as passed without running it.

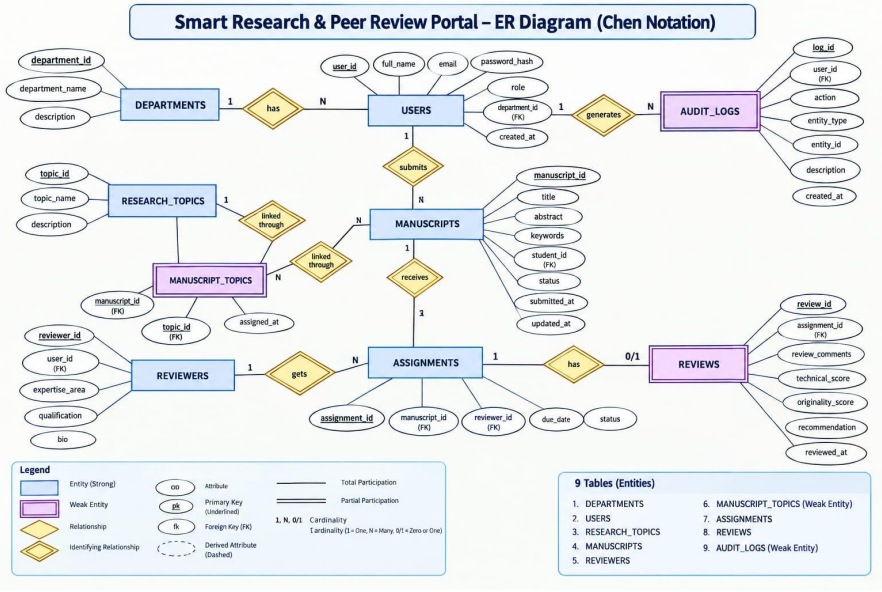
19. SCREENSHOTS AND EVIDENCE

Use only the following essential screenshots to keep the report focused. Crop each screenshot so that the relevant query, table, or application area is readable. Hide passwords, tokens, private email addresses, and other sensitive details.

Screenshot 1 – PostgreSQL table list

```
smartportal_db=# \dt
                List of tables
 Schema |       Name        | Type  |  Owner
--------+-------------------+-------+----------
 public | assignments       | table | postgres
 public | audit_logs        | table | postgres
 public | departments       | table | postgres
 public | manuscript_topics | table | postgres
 public | manuscripts       | table | postgres
 public | research_topics   | table | postgres
 public | reviewers         | table | postgres
 public | reviews           | table | postgres
 public | users             | table | postgres
(9 rows)
```

Screenshot 2 – ER diagram



Screenshot 3 – SQL JOIN output

```
smartportal_db=# SELECT
smartportal_db-#     m.manuscript_id,
smartportal_db-#     m.title AS manuscript_title,
smartportal_db-#     u.full_name AS student_name,
smartportal_db-#     u.email AS student_email,
smartportal_db-#     m.status
smartportal_db-# FROM manuscripts m
smartportal_db-# INNER JOIN users u
smartportal_db-# ON m.student_id = u.user_id
smartportal_db-# ORDER BY m.manuscript_id;
 manuscript_id |             manuscript_title              | student_name |    student_email     |    status
---------------+-------------------------------------------+--------------+----------------------+--------------
             1 | AI Based Crop Disease Detection           | Rahul Sharma | rahul@example.com    | UNDER_REVIEW
             2 | Smart Traffic Management System           | Ananya Rao   | ananya@example.com   | UNDER_REVIEW
             3 | IoT Based Smart Campus Monitoring System | Rahul Sharma | rahul@example.com    | SUBMITTED
```

.

Screenshot 4 – Trigger / audit evidence

```
               SELECT
smartportal_db-#     trigger_name,
smartportal_db-#     event_object_table,
smartportal_db-#     event_manipulation,
smartportal_db-#     action_timing
smartportal_db-# FROM information_schema.triggers
smartportal_db-# WHERE event_object_table = 'manuscripts';
          trigger_name          | event_object_table | event_manipulation | action_timing
--------------------------------+--------------------+--------------------+---------------
 manuscript_status_audit_trigger | manuscripts        | UPDATE             | AFTER
(1 row)

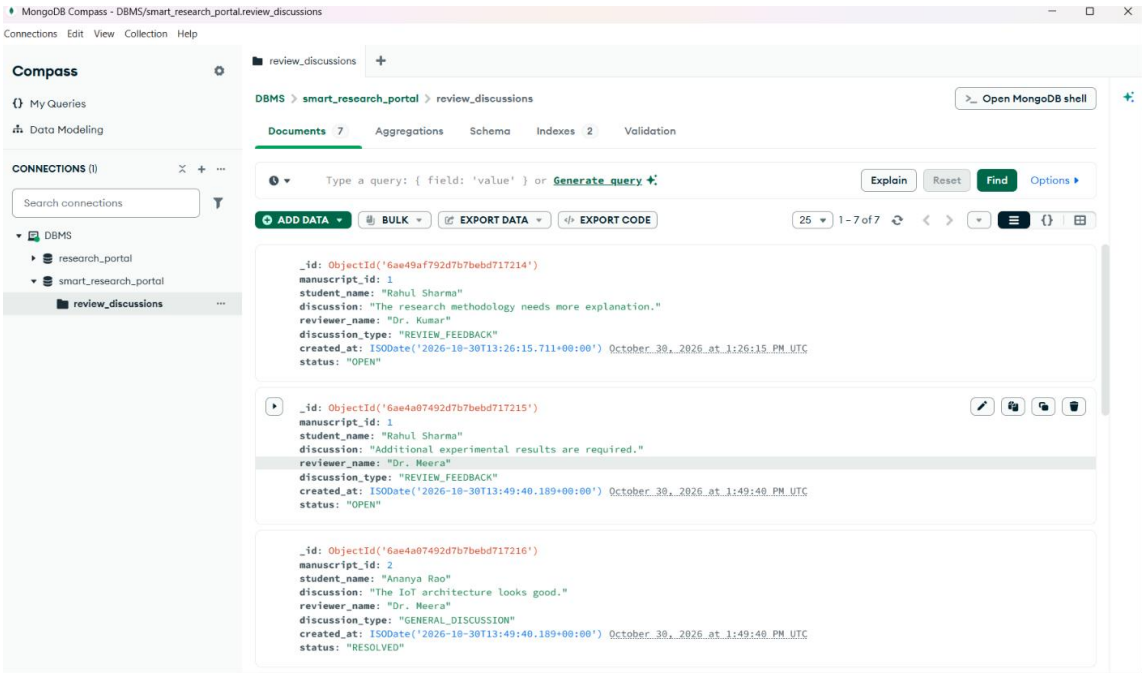

smartportal_db=# SELECT pg_get_functiondef(
smartportal_db(#     'public.log_manuscript_status_change()'::regprocedure
smartportal_db(# );
                      pg_get_functiondef
-------------------------------------------------------------
 CREATE OR REPLACE FUNCTION public.log_manuscript_status_change()+
  RETURNS trigger                                             +
  LANGUAGE plpgsql                                            +
 AS $function$                                                +
 BEGIN                                                        +
     INSERT INTO audit_logs (                                 +
         user_id,                                             +
         action,                                              +
         entity_type,                                         +
         entity_id,                                           +
         description                                          +
     )                                                        +
     VALUES (                                                 +
         NULL,                                                +
         'STATUS_CHANGED',                                    +
         'MANUSCRIPT',                                        +
         NEW.manuscript_id,                                   +
         'Manuscript status changed from '                    +
         || OLD.status || ' to ' || NEW.status                +
     );                                                       +
                                                              +
     RETURN NEW;                                              +
 END;                                                         +
 $function$                                                   +

(1 row)


smartportal_db=# SELECT *
smartportal_db-# FROM audit_logs
smartportal_db-# ORDER BY log_id;
 log_id | user_id |     action     | entity_type | entity_id |                    description                    |         created_at
--------+---------+----------------+-------------+-----------+---------------------------------------------------+----------------------------
      1 |         | STATUS_CHANGED | MANUSCRIPT  |         1 | Manuscript status changed from SUBMITTED to UNDER_REVIEW | 2026-10-30 19:11:45.174675
      2 |         | STATUS_CHANGED | MANUSCRIPT  |         2 | Manuscript status changed from SUBMITTED to UNDER_REVIEW | 2026-10-30 19:12:32.730249
(2 rows)


smartportal_db=#
```

Screenshot 5 MongoDB



20. EVALUATION RUBRICS – COMPLETION STATUS

The table below is a conservative status summary based on the implementation details available while preparing this report. Recheck each item in the final application and change the status only when verified.

Requirement / feature	Status to verify before submission	Evidence
Minimum five relational tables	Completed – nine physical tables were identified.	\dt output and ER diagram.
Relational database	Implemented – PostgreSQL database smartportal_db.	Database connection and table output.
SQL relationships and constraints	Implemented – primary keys, foreign keys, and unique constraints were identified.	Table definitions and ER diagram.
SQL JOIN operations	Implemented in the student review retrieval query.	API code and query output.
View	Existing view manuscript_review_summary was identified.	\dv and SELECT from view.
Function and trigger	Existing status-change function and trigger were identified.	Trigger/function definition and audit log.
Transactions	Concept and example documented; explicit application workflow not verified.	Show transaction code if implemented.
Student review dashboard	Frontend and API created; verify live retrieval of actual review records.	Running dashboard screenshot.
MongoDB / NoSQL integration	Not confirmed; verify database, collections, and application CRUD.	MongoDB Compass/mongosh and application code.

pgvector semantic search	Pending based on prior Windows application-control block.	Only mark complete after extension and query succeed.
Analytics/report output	Verify the actual report queries and displayed results.	Live report screenshot.

Do not claim all rubric items are complete simply because example SQL commands are included in this report. The final status must match the working project.

21. LIMITATIONS AND FUTURE ENHANCEMENTS

21.1 Current limitations to verify

- MongoDB application integration and collections need final verification.
- pgvector installation was blocked by Windows application-control policy.
- The audit function currently records a NULL user_id for status changes.
- The average of technical and originality scores is a calculated display value, not necessarily an official final grade.
- Pending reviewer assignments and submitted reviews are different states and should be shown separately.

21.2 Future enhancements

- Complete and demonstrate MongoDB-backed discussion or message storage if required.
- Enable and test pgvector for semantic manuscript or research-topic search.
- Record the authenticated actor in audit logs.
- Add notifications when a reviewer submits feedback.
- Define an official grading and multi-reviewer aggregation policy.
- Add more detailed analytics and downloadable review reports.
- Add automated tests for role-based access and student-specific data isolation.

22. CONCLUSION

The Smart Research & Peer Review Portal demonstrates how a relational database can support an academic manuscript-review workflow. PostgreSQL stores structured information about departments, users, research topics, manuscripts, reviewers, assignments, reviews, and audit logs. The Next.js application communicates with PostgreSQL through the pg connection pool and uses SQL queries to retrieve application data.

The student review feature demonstrates the use of relational joins, foreign-key relationships, parameterized queries, and session-based access control to present reviewer evaluations to the correct student. The database also demonstrates constraints, a view, a stored function, a trigger, and audit logging. The final report distinguishes confirmed implementation from planned or pending NoSQL and vector-search capabilities.

23. REFERENCES

- PostgreSQL Documentation – https://www.postgresql.org/docs/
- Next.js Documentation – https://nextjs.org/docs
- Node.js Documentation – https://nodejs.org/docs/
- node-postgres (pg) Documentation – https://node-postgres.com/
- MongoDB Documentation – https://www.mongodb.com/docs/
- jose documentation – https://github.com/panva/jose

END OF REPORT